API文档

公共类型

`agv::mq_msg` 接口*

概述

mq_msg.h 定义了进程间消息队列使用的消息格式、队列参数、优先级和 MQTT 发布消息构造方式。

- 适用于 `/mq_task_dispatch` 和 `/mq_mqtt_publish`
- 提供固定长度消息结构
- 包含 MQTT 主题/负载生成辅助函数

常量

系统规模与缓冲区

- `kMaxNodes`
  - 系统最大节点数
  - 由 `AGV_MAX_NODES` 配置决定
- `kTopicMaxLen = 128`
  - MQTT topic 最大长度（含终结符 `\0`）
- `kPayloadMaxLen = 256`
  - MQTT payload 最大长度（含终结符 `\0`）
- `kLogMsgMaxLen = 512`
  - 日志消息最大长度（含终结符 `\0`）
- `kProcNameMaxLen = 32`
  - 进程名最大长度（含终结符 `\0`）

消息队列名称

- `kMqTaskDispatch = "/mq_task_dispatch"`
- `kMqMqttPublish  = "/mq_mqtt_publish"`

队列容量参数

`/mq_task_dispatch`

- `kTaskDispatchMaxMsg = 16`
- `kTaskDispatchMsgSize = 256`

`/mq_mqtt_publish`

- `kMqttPublishMaxMsg = 32`
- `kMqttPublishMsgSize = 512`

消息优先级

```
enum MsgPriority : unsigned {
    kPrioLow    = 0,
    kPrioNormal = 1,
    kPrioHigh   = 2,
};
```

- 数字越大优先级越高
- `kPrioHigh` 适用于紧急命令、取消任务等

任务调度消息

`/mq_task_dispatch`

调度类型

`enum class TaskAction : uint8_t`

- `kAssign = 0` 新建任务
- `kCancel = 1` 取消任务
- `kReplan = 2` 重规划

调度结构体

`struct TaskDispatchMsg`

字段

- `TaskAction action`
- `uint8_t car_id`
  - 小车 ID
  - `0xFF` 表示广播或任意车
- `uint8_t target_node`
  - 目标节点 ID
  - `kCancel` 时忽略
- `uint8_t immediate`
  - `1` = 立即生效
  - `0` = 等当前段完成后生效

构造函数

- `TaskDispatchMsg::assign(uint8_t car, uint8_t target, bool imm = true)`
- `TaskDispatchMsg::cancel(uint8_t car)`
- `TaskDispatchMsg::replan(uint8_t car)`

MQTT 发布消息

`/mq_mqtt_publish`

命令类型

`enum class MqttCmdType : uint8_t`

- `CMD_angle = 0`
- `CMD_ori = 1`
- `QUERY = 2`
- `ACTION = 3`

命令参数结构

/与类型对应

- `struct AngleParam`
  - `uint16_t angle`
  - padded bytes
- `struct OriParam`
  - `OriCmd cmd`（见枚举）
- `struct QueryParam`
  - `QueryCmd cmd`（见枚举）
- `struct ActionParam`
  - `ActionCmd cmd（见枚举）`

命令枚举

- `enum class OriCmd : uint8_t`
  - `kStraight = 0`
  - `kLeft = 1`
  - `kRight = 2`
- `enum class QueryCmd : uint8_t`
  - `kStatus = 0`
  - `kLog = 1`
- `enum class ActionCmd : uint8_t`
  - `kPause = 0`
  - `kProcess = 1`

- `kReboot = 2`

发布结构体

`struct MqttPublishMsg`

字段

- `MqttCmdType cmd_type`
- `uint8_t car_id`
- `uint8_t next_node`
  - 仅在移动类命令使用
- `uint8_t qos`
- `int64_t timestamp_us`
- `union params`
  - `AngleParam c_angle`
  - `OriParam c_ori`
  - `QueryParam c_query`
  - `ActionParam c_action`

便捷构造函数

- `make_angle(uint8_t car_id, uint16_t angle)`
- `make_ori(uint8_t car_id, OriCmd cmd)`
- `make_query(uint8_t car_id, QueryCmd cmd)`
- `make_action(uint8_t car_id, ActionCmd cmd)`

MQTT 转换函数

```
void to_mqtt(char* topic_buf,
             size_t topic_buf_size,
             char* payload_buf,
             size_t payload_buf_size) const;
```

- 生成 MQTT topic，例如：`agv/car/<car_id>/cmd/<cmd_type>`
- 生成 MQTT payload，当前实现为简单 JSON 风格字符串
- 如果 `cmd_type` 不识别，则 `payload_buf[0] = '\0'`

系统服务层

log-日志模块

调用

头文件引入

```
#include "agv/log_msg.h"
#include "logger.h"
```

cmake连接

```
target_link_libraries(xxx PRIVATE agv_logger xxx)
```

函数接口

准备工作

```
//mq是信号队列的句柄，保留
mqd_t mq = agv_log_init();
```

收尾工作

```
mq_close(mq);
```

推荐函数接口/宏定义

```

// Convenience macros for logging at different levels.
#define LOG_DEBUG(src, fmt, ...) agv_logf(agv_log_init(), LogLevel::DEBUG, src,
fmt, ##__VA_ARGS__)
#define LOG_INFO(src, fmt, ...)  agv_logf(agv_log_init(), LogLevel::INFO, src,
fmt, ##__VA_ARGS__)
#define LOG_WARN(src, fmt, ...)  agv_logf(agv_log_init(), LogLevel::WARN, src,
fmt, ##__VA_ARGS__)
#define LOG_ERROR(src, fmt, ...) agv_logf(agv_log_init(), LogLevel::ERROR, src,
fmt, ##__VA_ARGS__)
#define LOG_FATAL(src, fmt, ...) agv_logf(agv_log_init(), LogLevel::FATAL, src,
fmt, ##__VA_ARGS__)
```

日志查看

终端查看

可以在`include/config/config.h`配置是否直接终端打印信息。

```
//是否打印注释，若不直接打印，请注释
#define _AGV_PRINT_DEBUG
```

系统查找日志

```
# 实时跟踪 agv 日志
journalctl -f SYSLOG_IDENTIFIER=agv_log

# 只看 error 以上（PRIORITY <= 3）
journalctl SYSLOG_IDENTIFIER=agv_log -p err

# 按模块过滤（用自定义字段）
journalctl SYSLOG_IDENTIFIER=agv_log AGV_SOURCE=planner

# 看带时间戳的完整输出
journalctl SYSLOG_IDENTIFIER=agv_log -o verbose
```

信号处理框架*

调用

头文件引入

```
#include "signal_handler.h"
#include "secure_exit.h"
```

cmake连接

```
target_link_libraries(xxx PRIVATE agv_ipc xxx)
```

函数接口

类型定义

```
using Usr1Callback = void (*)(const char* proc_name);
```

- SIGUSR1 触发时调用的回调类型
- 参数为进程名字符串

`agv::SignalHandler` 接口

阻塞内核的异步信号 `SIGTERM`、`SIGINT`、`SIGUSR1`。防止其打断重要操作。其可由 `poll()` 监听。

构造函数

```
explicit SignalHandler(const char* proc_name = "agv_proc",
                       Usr1Callback usr1_cb = nullptr);
```

- `proc_name`
  - 进程名称，用于日志或回调标识
- `usr1_cb`
  - SIGUSR1 到来时执行的回调
  - 传 `nullptr` 表示不处理 SIGUSR1

析构函数

```
~SignalHandler();
```

- 关闭 `signalfd` 文件描述符
- 若 `sfd_ >= 0`，调用 `::close(sfd_)`

禁止拷贝

- `SignalHandler(const SignalHandler&) = delete;`
- `SignalHandler& operator=(const SignalHandler&) = delete;`

初始化信号处理组件

```
void init();
```

- 创建一个包含 `SIGTERM`、`SIGINT`、`SIGUSR1` 的信号集
- 使用 `sigprocmask(SIG_BLOCK, &mask_, nullptr)` 阻塞这三个信号
- 创建 `signalfd`，并设置 `SFD_NONBLOCK | SFD_CLOEXEC`

异常：

- `std::runtime_error` 当 `sigprocmask` 或 `signalfd` 调用失败时抛出

> 该方法必须在进入主循环之前调用，通常在 `main()` 开头。

查询文件描述符

```
int get_fd() const;
```

返回 `signalfd` 的文件描述符。

- 供 `poll()` 或 `epoll` 注册使用
- 必须在 `init()` 之后调用

注册退出/用户事件处理

`void handle_read();`

处理 `signalfd` 可读事件。

- 循环读取所有待处理信号
- 对 `SIGTERM` 和 `SIGINT`：设置内部 shutdown 标志
- 对 `SIGUSR1`：如果注册了 `usr1_cb_`，则调用回调

注意：

- `sfd_` 使用非阻塞模式
- 读取结束条件为 `EAGAIN` / `EWOULDBLOCK` 或读到不完整数据

查询退出

```
bool shutdown_requested() const;
```

查询是否已收到退出信号。

- 返回内部原子布尔 `g_shutdown_`
- 主要用于主循环退出条件
- 使用 `memory_order_acquire` 保证可见性

查询进程名

```
const char* proc_name() const;
```

返回进程名字符串。

`agv::SecureExit` 接口文档

它用于在主循环结束后，按固定顺序执行清理动作，保证资源正确关闭并通知 systemd。

- 逆序执行注册的清理回调
- 可选等待当前任务完成
- 支持 systemd 通知 `STOPPING=1`
- 直接调用 `exit(0)` 结束进程

构造函数

```
explicit SecureExit(const char* proc_name);
```

- `proc_name`
  - 进程名称，用于日志输出
  - 样例："planner"

内部类型说明

```
struct CleanupEntry {
    std::string           name;
    std::function<void()> fn;
};
```

- `name`
  - 清理动作的名称
  - 仅用于日志描述
- `fn`
  - 清理回调函数
  - 支持任意可调用对象

注册退出函数

```
void add_cleanup(std::string name, std::function<void()> fn);
```

注册退出时需要执行的清理动作。

- `name`
  - 清理条目的描述
- `fn`
  - 执行清理的函数

执行顺序：LIFO

执行退出

```
[[ noreturn ]] void run(int wait_ms = 200);
```

执行完整退出序列并结束进程。

- `wait_ms`
  - 等待当前任务完成的最长毫秒数
  - `0` 表示不等待
  - 默认值：`200`

退出序列步骤：

1. 输出启动退出序列日志
2. 如果 `wait_ms > 0`，休眠指定时间
3. 逆序执行所有注册的清理回调
4. 调用 `sd_notify(0, "STOPPING=1")` 通知 systemd
   - 如果未启用 systemd，则宏定义 `AGV_NO_SYSTEMD` 会让此调用成为空实现
5. 调用 `::exit(0)` 终止进程

异常处理：

- 如果清理回调抛出 `std::exception`，会记录错误日志
- 如果抛出非标准异常，也会记录未知异常日志
- 退出过程继续执行后续回调

写入独立日志

```
void log_info(const std::string& msg);
```

- 将信息日志直接写入 `stderr`
- 不依赖外部日志模块

```
void log_err(const std::string& msg);
```

- 将错误日志直接写入 `stderr`

业务消息队列

调用

头文件引入

```
#include "mq_wrapper.h"
```

cmake连接

```
target_link_libraries(xxx PRIVATE agv_ipc xxx)
```

管理程序接口 `agv::MqOwner`

- 负责创建和销毁队列
- 适合系统启动时的队列初始化
- 析构时自动 `unlink` 未删除的队列

准备工作

```
void create(const char* name, long max_msg, long msg_size);
```

- 创建队列
- 特点
  - 先 `mq_unlink` 旧队列，避免旧队列遗留参数不一致
  - 使用 `O_CREAT | O_RDWR`
  - 创建后立即 `mq_close`
- 异常
  - `std::runtime_error`：创建失败

```
void create_all();
```

- 创建系统所需的全部队列
- 当前实现创建
  - `kMqTaskDispatch`
  - `kMqMqttPublish`

收尾工作

```
void unlink_all();
```

- 删除所有已记录队列
- 释放资源，清空队列列表

接收程序接口 `agv::MqReceiver<Msg>`

- 只读端消费者使用
- 若队列不存在则创建
- 不会在 `receive()` 内部阻塞

准备工作

```
void init(const char* name, long max_msg = 16, long msg_size = sizeof(Msg));
```

- 打开或创建队列
- 参数
  - `name`：队列名，例如 `"/mq_task_dispatch"`
  - `max_msg`：队列最大消息数
  - `msg_size`：单条消息最大字节数
- 要求
  - `msg_size >= sizeof(Msg)`
- 异常
  - `std::runtime_error`：`mq_open` 失败
  - `std::runtime_error`：消息尺寸不够

服务接口

```
bool receive(Msg& msg, unsigned& prio);
```

- 从队列读取一条消息
- 参数
  - `msg`：输出接收结构体
  - `prio`：输出消息优先级
- 返回值
  - `true`：成功读取一条消息
  - `false`：队列为空（`EAGAIN`）或发生错误
- 特点
  - 非阻塞
  - 出错时打印 stderr，不抛出

```
const char* name() const;
```

- 返回队列名称

发送函数接口`agv::MqSender<Msg>`

- 只写端生产者使用
- 队列必须已由接收端或 `MqOwner` 创建
- `msg_size` 需要显式传入

准备工作

```
void init(const char* name, long msg_size = sizeof(Msg));
```

- 打开已存在队列
- 参数
  - `name`：队列名
  - `msg_size`：队列 msg_size，建议使用对应常量
- 异常
  - `std::runtime_error`：`mq_open` 失败

服务接口

```
bool send(const Msg& msg, unsigned prio = kPrioNormal);
```

- 发送一条消息
- 参数
  - `msg`：要发送的消息数据
  - `prio`：使用 `MsgPriority` 枚举
- 返回值
  - `true`：发送成功
  - `false`：队列已满或发送失败
- 异常
  - `std::runtime_error`：
    - 如果 `sizeof(Msg) > msg_size_`
- 错误处理
  - `EAGAIN`：队列满，打印警告并返回 `false`
  - 其他错误：打印错误并返回 `false`

```
const char* name() const;
```

- 返回队列名称

模型存储层

注意，在这之前还有seqlock服务，在这里略去。

SHM管理器

- `ShmOwner`：由 `model_manager` 进程持有，负责创建、初始化和销毁共享内存。
- `ShmClient`：由其他消费者进程持有，负责 attach 到已存在的共享内存并校验结构。

调用

CMAKE连接

```
target_link_libraries(xxx PRIVATE agv_ipc xxx)
```

头文件使用

```
#include "shm_manager.h"
```

`agv::ShmBase`(仅参考)

基础类，封装共享内存映射的生命周期管理。

公开接口：

- `ShmLayout* ptr()`
  - 返回映射后的 `ShmLayout*`，供调用方访问共享内存结构。
- `const ShmLayout* ptr() const`
- `bool is_attached() const`
  - 判断当前是否已 attach 或创建成功。

内部行为：

- 析构时自动调用 `munmap`。
- 禁止拷贝和赋值。

管理程序接口`agv::ShmOwner`

由 owner 进程使用，负责创建并初始化共享内存。

准备工作

```
void create_and_init()
```

- 创建共享内存对象并完成初始化。
- 步骤：
  1. `shm_unlink(kShmName)` 清理残留
  2. `shm_open(O_CREAT | O_RDWR, 0666)`
  3. `ftruncate` 设定大小
  4. `mmap` 映射
  5. placement new 初始化 `Seqlock`

    6. value-initialize 清零 `MapData` / `CarData`
    7. 写入 `ShmHeader`，最后设置 `initialized = 1`
- 异常：
  - `std::runtime_error`：创建失败、`ftruncate` 失败、`mmap` 失败 等

收尾工作

自动析构，释放映射。

使用约定：

- 仅应在单一 owner 进程中调用一次。
- `ptr()` 返回的共享内存指针在对象生命周期内有效。

用户接口 `agv::ShmClient`

由其他进程使用，负责附加到已存在的共享内存并校验结构。

准备工作

```
void attach(int retry_ms = 500)
```

- attach 到已存在的共享内存并校验 header。
- 参数：
  - `retry_ms`：等待 `initialized` 标志的最长毫秒数，`0` 表示不等待。
- 操作：
  1. `shm_open(kShmName, O_RDWR, 0)`
  2. `mmap`
  3. `validate_header(retry_ms)`
- 异常：
  - `std::runtime_error`：SHM 不存在、`mmap` 失败、初始化超时、magic/version/size 不匹配
- `~ShmClient()`
  - 基类析构时自动 `munmap`，但不 `shm_unlink`。

私有方法：

- `validate_header(int retry_ms)`
  - 等待 `header.initialized`，再检查：
    - `magic`
    - `version`
    - `shm_size`

使用约定：

- 其他进程应先调用 `attach()`，再使用 `ptr()` 访问共享内存。
- 可通过 `retry_ms` 等待 owner 完成初始化。

服务函数（非类成员）

MapData读（全部）

```
MapData shm_read_map(ShmLayout* shm)
```

- 读取整块 `MapData` 快照。
- 返回本地副本，调用方不再持有锁。

MapData读（单条）

```
Edge shm_read_edge(ShmLayout* shm, uint16_t edge_idx)
```

- 读取单条 `Edge` 快照。
- `edge_idx` 为数组下标，非 `edge.id`。

MapData写（单条）

```
void shm_set_edge_status(ShmLayout* shm, uint16_t edge_idx, EdgeStatus status)
```

- 修改单条边状态。

CarData读（全部）

```
CarData shm_read_cars(ShmLayout* shm)
```

- 读取整块 `CarData` 快照。

CarData读（单条）

```
Car shm_read_car(ShmLayout* shm, uint8_t car_idx)
```

- 读取单辆车状态。
- `car_idx` 为数组下标，非 `car.id`。

CarData写（单条）

```
void shm_update_car(ShmLayout* shm, uint8_t car_idx, const Car& car)
```

- 更新单辆车完整状态。

CarData写（单条状态）

```
void shm_set_car_status(ShmLayout* shm, uint8_t car_idx, CarStatus status,
uint8_t current_node)
```

- 仅更新车的状态字段和当前位置节点 ID。